

ForestLens: counting tree crowns and bounding canopy area

Approach, architecture, key decisions, what worked, what didn't, and known limitations. Flora Carbon AI hiring hackathon submission.
Ravi Yadav · Live demo: forestlens-cujmcn43y5s8ev8vguauhez.streamlit.app · Source, data and full report: github.com/RAVIYADAV6522/forestlens

1 What it does

Upload high-resolution forest imagery and ForestLens returns individual tree-crown detections drawn over your image, a tree count, per-crown areas in square metres, a canopy-cover estimate, and downloadable results — with the assumptions and failure cases on screen rather than in a footnote. It is a Streamlit application with three licence-verified sample scenes bundled, so a stranger can run it in one click.

2 Architecture and workflow

imagery → read GSD/CRS from the raster → resample to the detector's training resolution → windowed tiled detection → boxes + confidence → IoU duplicate suppression → edge flags → map boxes back to source pixels → optional crown masks → pixel area × GSD² → count, crown area, cover interval → annotated PNG, CSV, GeoJSON, run metadata.

- **Counting and area are separate stages.** The count comes from the detector; the area comes from whatever crown geometry is defensible for that imagery.
- **Detection runs on a scale-matched copy**, and boxes are mapped back into source-raster pixels, so annotation, area and geospatial export share one coordinate frame.
- **Imagery is window-addressed, not downloaded.** Fetch scripts read only the requested window from the provider's cloud-optimised GeoTIFF, so each crop is 1–10 MB and rebuilds exactly from a scene ID and pixel window.
- **Every run exports its own metadata** — image identity, GSD and its origin, model versions, all thresholds, resampling scale, timings, peak memory — so any count is reproducible from its own export.

3 Key technical decisions

Resample to the model's training resolution, anchored not tuned. The pretrained detector was trained on ~0.10 m/px imagery. On 0.305 m satellite imagery it returned 5.8 trees/ha with 15 m apparent crowns where real crowns are 4–8 m. Resampling the detection input to 0.10 m takes the same scene to 71.2 trees/ha with 5.2 m crowns. The scale factor is fixed to the documented training GSD, because the count rises monotonically with scale and never plateaus — so picking the scale with the nicest count would be tuning to a preferred answer. Resampling adds no information and the app says so.

Cover from the union of crowns, not the sum. Summing crown boxes double-counts overlaps. Cover uses the union of footprints divided by analysed ground area.

Report cover as an interval, and refuse a point estimate. Lower bound: the exact geometric union of crown footprints (*shapely.ops.unary_union*) — dissolved rather than rasterised, because rasterising fractional boxes counts every pixel touched and inflated this by ~2 points. Upper bound: green-vegetation cover ($2G - R - B > 0$), or optionally a semantic model. Beyond 25 points apart, the app states the range and declines a single figure.

Withhold rather than assume. No GSD means no physical area — pixel counts only. No detector installed means the app refuses to count rather than inventing a number; there is deliberately no synthetic fallback detector.

Verify the library, don't trust it. The detector's tiled-prediction docstring specifies BGR input; the pipeline passes RGB. Tested both on the library's own in-distribution crop: RGB gave 55 detections at mean confidence 0.535, BGR gave 29 at 0.389. RGB is correct and the docstring is stale. Had it gone the other way, every count here would have come from transposed colour channels.

4 What worked

Scene	Native GSD	Trees	/ha	Cover interval
S1a open (Maxar WV-02)	0.49 m	695	71.2	15.1–92.6%
S1b dense (Maxar WV-02)	0.49 m	705	72.2	13.3–96.3%
S2 closed (SWISSIMAGE)	0.10 m	158	37.7	21.0–93.9%

- **End-to-end on real licensed imagery**, 6–14 s per scene, with annotated output, CSV, GeoJSON and run metadata.
- **Scale matching turned an unusable detector into a working one** on satellite imagery (§3).
- **Scale accounting, after a defect**: DeepForest rescales every tile to 800 px internally, so the tile-size control was silently changing the detector's scale and fragmenting crowns (400 px tile: 1379 trees, 2.1 m median crowns; 800 px: 695 trees, 4.2 m). That factor is now divided out and median crown width is tile-invariant.
- **Negative controls behaved**. Open water: 2 detections over 9.77 ha, 0.0% crown union. Urban core: detections land on real street trees with building roofs almost entirely clear.
- **124 tests and 13 mechanical QA checks** pass without model weights, so the measurement arithmetic is verifiable without a GPU.

5 What didn't work

Closed canopy breaks the approach. Scene S2 is at exactly the detector's training resolution and is the worst performer of the three: on a counted sub-crop it recovered 9 crowns against a reference of 48 — about one in five — and reports 21% cover on canopy that is visibly 95–100% closed. Per-hectare counts also saturate: 71.2 against 72.2 on two visibly different stand densities. **Canopy closure, not resolution, is the binding constraint**, which inverts the instinct that finer imagery fixes individual-tree analysis.

Open canopy over-counts by 19–28% against counted crops, most likely clump splitting and shadow detections.

Canopy cover cannot be measured to useful precision. The two bounds sit 73–81 points apart on all three scenes. Crown detection answers “how many trees”; a canopy-versus-ground classifier answers “how much cover”, and using the first for the second is wrong by a factor of four under closure.

A scale-accounting defect of my own, found late. DeepForest rescales every tile to an 800 px short side, so the operating scale is $\text{patch_size} \times \text{gsd} / 800$ — which this pipeline accounted for nowhere. The tile-size control was therefore a hidden scale control, moving the count 3.4x and the median detected crown 2.0x (at a 400 px tile, crowns were reported at 2.1 m against a real 4–8 m, i.e. fragmented into 2–4 boxes each). Now divided back out; a strict no-op at the shipped default, so every figure here stands. It also forced a correction: part of what I documented as a closed-canopy failure was this scale choice, so the claim that closure rather than resolution is the binding constraint does not survive — see the report. Separately, my own IoU suppression control removes zero boxes, because DeepForest already bounds every output pair below its internal 0.15.

Neither cover bound is trustworthy. The vegetation index is fooled by anything green: 94.3% “vegetation” on open lake water. An optional SegFormer (ADE20K tree/plant) is far better on exactly those controls — water 29.3%, urban 1.7% against 31.0% — and comparable on closed canopy, so it ships as an option. Not as the default: it is scale-sensitive on nadir imagery, reading 86.2% whole-image against 1.6% tiled on one scene, and it saturates at 100% on dense conifer. Whichever produced the bound is recorded in the run metadata.

Two fixes I built and threw away. Otsu thresholding for the vegetation bound splits within the vegetation distribution on a uniformly vegetated image and claimed 62% cover on the ~100%-closed scene. Texture filtering, to stop turbid water reading as 94.3% vegetation, failed because water sits at median σ 4.8 and closed canopy at 20.0, but the open stand is 6.3 — any threshold removing the lake also deletes a real forest scene. The app flags the condition instead of shipping a fix that only works on one image.

6 Known limitations and shortcuts

The reference counts behind “+19–28%” and “one in five” were made by eye by the AI assistant that implemented the system — not by a human expert and not by field survey. They are independent of the detector but they are not ground truth, and both headline figures inherit that weakness. Replacing them is the single most valuable extension of this work.

- **No accuracy figure is published anywhere.** There is no labelled evaluation set for these scenes, and inspecting an overlay does not yield precision or recall. Reported confidences are model scores, labelled as such.

- **Three scenes, two providers, two forest types.** The closure finding rests on one 0.10 m scene, and closure is confounded with structural domain shift — the detector's training data is largely open North American conifer, not closed European mixed forest.
- **Crown masks are excluded from the deployed build.** SAM 2 works locally (0.12 s/crown) and shows boxes overestimate crown area by 12–22%, but it loads a second model and does not touch the dominant error. Deployed areas are bounding-box proxies, labelled as such.
- **No fine-tuning.** Pretrained weights only, which is why closed canopy fails; no threshold setting reaches an 80% miss rate.
- **The upper cover bound is an approximation, not a guarantee.** It over-counts non-tree vegetation and green water, under-counts shadowed canopy, and moves from 92.6% to 65.7% on one scene depending on the threshold.
- **Geometry is uncorrected.** Crowns clipped by the image edge have truncated areas and are flagged per crown (30 of 695 on S1a). Off-nadir view geometry and terrain relief are not corrected for.
- **Imagery licensing constrains reuse.** The Maxar scenes are CC BY-NC — non-commercial — so productising this would need separately licensed imagery. The Swiss scene permits commercial use with attribution.
- **Generalisation is untested**, including on Indian forests, which are structurally closer to the scene where the method failed worst. It would need fine-tuning and local validation before anyone trusted a number from it.
- **No carbon or biomass estimate** — deliberately. Canopy area is geometry; converting it needs allometry, species and field calibration this work does not have. For a carbon-market application, declining that conversion is a correctness property, not a missing feature.